The Stack ADT

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Definition
2. Operations
3. Applications
4. Array Implementation
5. Linked-List Implementation
6. Stack Applications

Definition and Properties

- A **stack** is a linear ADT supporting access at one end (**top**).
- **LIFO** discipline: Last-In, First-Out.
- **State:** an ordered sequence $\langle a_1, \ldots, a_k \rangle$ where a_k is at the top.
- **Safety:** operations defined only when preconditions hold (e.g., non-empty for `top`/`pop`).
- **Common capacities:** bounded (array-based) or unbounded (linked-list).

Basic Operations

- `push()`: insert item at top.
- `pop()`: remove and return top element.
- `top()`: return (peek) top element without removing.
- `isEmpty()`: true if stack has no elements.
- `size()`: number of elements.

Exceptions

- **Underflow:** calling `pop`/`top` on empty stack.
- **Overflow:** pushing onto a full bounded stack (array capacity reached).

Applications of Stacks

- **Expression processing:** infix $\leftrightarrow$ postfix conversion, evaluation.
- **Syntax parsing:** balanced parentheses, compiler front-ends.
- **Backtracking:** DFS, path exploration, undo operations.
- **Function call management:** activation records, recursion.
- **Memory management:** temporary storage, local variables, return addresses.

Stack Application: C++ Runtime Stack

- Each function call creates an **activation record** (stack frame) storing:
  - return address, parameters, local variables, saved registers.
- **Call** $\rightarrow$ **push frame**, **return** $\rightarrow$ **pop frame**.
- Enables recursion: each recursive call has its own frame (separate locals).
- Stack overflow occurs if recursion is too deep or frames are too large.

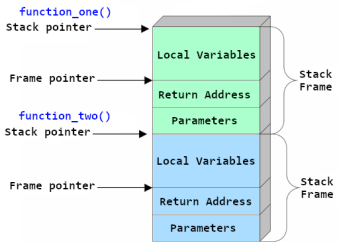


Figure: Figure credit goes to Nexus Software Systems

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$
5	`pop()`	9	$S = \langle 7, 3 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$
5	`pop()`	9	$S = \langle 7, 3 \rangle$
6	`push(4)`	void	$S = \langle 7, 3, 4 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$
5	`pop()`	9	$S = \langle 7, 3 \rangle$
6	`push(4)`	void	$S = \langle 7, 3, 4 \rangle$
7	`size()`	3	$S = \langle 7, 3, 4 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$
5	`pop()`	9	$S = \langle 7, 3 \rangle$
6	`push(4)`	void	$S = \langle 7, 3, 4 \rangle$
7	`size()`	3	$S = \langle 7, 3, 4 \rangle$
8	`pop()`	4	$S = \langle 7, 3 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$
5	`pop()`	9	$S = \langle 7, 3 \rangle$
6	`push(4)`	void	$S = \langle 7, 3, 4 \rangle$
7	`size()`	3	$S = \langle 7, 3, 4 \rangle$
8	`pop()`	4	$S = \langle 7, 3 \rangle$
9	`pop()`	3	$S = \langle 7 \rangle$

Example 1: Stack Operations (Trace)

Initial state: $S = \langle\ \rangle$ (empty)

Step	Operation	Return	New State
1	`push(7)`	void	$S = \langle 7 \rangle$
2	`push(3)`	void	$S = \langle 7, 3 \rangle$
3	`top()`	3	$S = \langle 7, 3 \rangle$
4	`push(9)`	void	$S = \langle 7, 3, 9 \rangle$
5	`pop()`	9	$S = \langle 7, 3 \rangle$
6	`push(4)`	void	$S = \langle 7, 3, 4 \rangle$
7	`size()`	3	$S = \langle 7, 3, 4 \rangle$
8	`pop()`	4	$S = \langle 7, 3 \rangle$
9	`pop()`	3	$S = \langle 7 \rangle$
10	`isEmpty()`	`false`	$S = \langle 7 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$
3	`push(50)`	void	$S = \langle 10, 20, 30, 40, 50 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$
3	`push(50)`	void	$S = \langle 10, 20, 30, 40, 50 \rangle$
4	`size()`	5	$S = \langle 10, 20, 30, 40, 50 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$
3	`push(50)`	void	$S = \langle 10, 20, 30, 40, 50 \rangle$
4	`size()`	5	$S = \langle 10, 20, 30, 40, 50 \rangle$
5	`push(60)`	**Error: Overflow**	$S = \langle 10, 20, 30, 40, 50 \rangle$ (state unchanged)

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$
3	`push(50)`	void	$S = \langle 10, 20, 30, 40, 50 \rangle$
4	`size()`	5	$S = \langle 10, 20, 30, 40, 50 \rangle$
5	`push(60)`	**Error: Overflow**	$S = \langle 10, 20, 30, 40, 50 \rangle$ (state unchanged)
6	`pop()`	50	$S = \langle 10, 20, 30, 40 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10, 20, 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$
3	`push(50)`	void	$S = \langle 10, 20, 30, 40, 50 \rangle$
4	`size()`	5	$S = \langle 10, 20, 30, 40, 50 \rangle$
5	`push(60)`	**Error: Overflow**	$S = \langle 10, 20, 30, 40, 50 \rangle$ (state unchanged)
6	`pop()`	50	$S = \langle 10, 20, 30, 40 \rangle$
7	`push(70)`	void	$S = \langle 10, 20, 30, 40, 70 \rangle$

Example 2: Stack Operations (Trace)

Initial state: $S = \langle 10,\ 20,\ 30 \rangle$ (size $= 3$, capacity $= 5$)

Step	Operation	Return	New State
1	`top()`	30	$S = \langle 10, 20, 30 \rangle$
2	`push(40)`	void	$S = \langle 10, 20, 30, 40 \rangle$
3	`push(50)`	void	$S = \langle 10, 20, 30, 40, 50 \rangle$
4	`size()`	5	$S = \langle 10, 20, 30, 40, 50 \rangle$
5	`push(60)`	**Error: Overflow**	$S = \langle 10, 20, 30, 40, 50 \rangle$ (state unchanged)
6	`pop()`	50	$S = \langle 10, 20, 30, 40 \rangle$
7	`push(70)`	void	$S = \langle 10, 20, 30, 40, 70 \rangle$
8	`isEmpty()`	`false`	$S = \langle 10, 20, 30, 40, 70 \rangle$

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$
3	`push(88)`	void	$S = \langle 42, 17, 88 \rangle$

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$
3	`push(88)`	void	$S = \langle 42, 17, 88 \rangle$
4	`pop()`	88	$S = \langle 42, 17 \rangle$

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$
3	`push(88)`	void	$S = \langle 42, 17, 88 \rangle$
4	`pop()`	88	$S = \langle 42, 17 \rangle$
5	`pop()`	17	$S = \langle 42 \rangle$

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$
3	`push(88)`	void	$S = \langle 42, 17, 88 \rangle$
4	`pop()`	88	$S = \langle 42, 17 \rangle$
5	`pop()`	17	$S = \langle 42 \rangle$
6	`pop()`	42	$S = \langle\ \rangle$ (empty)

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$
3	`push(88)`	void	$S = \langle 42, 17, 88 \rangle$
4	`pop()`	88	$S = \langle 42, 17 \rangle$
5	`pop()`	17	$S = \langle 42 \rangle$
6	`pop()`	42	$S = \langle\ \rangle$ (empty)
7	`pop()`	**Error: Underflow**	$S = \langle\ \rangle$ (state unchanged)

Example 3: Stack Operations (Trace)

Initial state: $S = \langle 42,\ 17 \rangle$ (size $= 2$, capacity $= 5$)

Step	Operation	Return	New State
1	`size()`	2	$S = \langle 42, 17 \rangle$
2	`top()`	17	$S = \langle 42, 17 \rangle$
3	`push(88)`	void	$S = \langle 42, 17, 88 \rangle$
4	`pop()`	88	$S = \langle 42, 17 \rangle$
5	`pop()`	17	$S = \langle 42 \rangle$
6	`pop()`	42	$S = \langle\ \rangle$ (empty)
7	`pop()`	**Error: Underflow**	$S = \langle\ \rangle$ (state unchanged)
8	`isEmpty()`	`true`	$S = \langle\ \rangle$

Stack via Array: Declaration & Idea

- A stack can be implemented using a fixed-size array.
- We keep track of the index of the current **top** element using an integer variable.
- Initially, when the stack is empty, top is set to `-1`.
- The stack is considered **full** when top reaches `capacity-1`.

Stack via Array: Basic Operations

- **Push:** Increase top index by one and insert the new element at that position. If the stack is already full, an **overflow** error occurs.
- **Pop:** Remove the element at the top index and decrease top by one. If the stack is empty, an **underflow** error occurs.
- **Top/Peek:** Access the element at the current top index without removing it.
- **isEmpty:** Check whether top is `-1`.
- **Size:** The number of elements is always `top + 1`.

Array Stack: Complexity & Pros/Cons

- **Time Complexity:** `push`, `pop`, `top`, `isEmpty`, and `size` all run in $\Theta(1)$ time.
- **Space:** Requires $\Theta(\text{capacity})$ contiguous memory for the array.

Pros

- Simple and efficient implementation.
- Excellent cache locality and predictable memory usage.

Cons

- Fixed capacity; cannot exceed the pre-allocated size.
- Resizing (if allowed) requires allocating new memory and copying all elements ($O(n)$).

Stack via Linked List: Declaration & Idea

- A stack can also be implemented using a **singly linked list**.
- Each node stores:
  - a **value**, and
  - a pointer to the **next** node.
- The stack maintains a pointer `head` that always points to the top node.
- The stack is **empty** when `head = nullptr`.

Stack via Linked List: Basic Operations

- **Push:** Create a new node, set its next pointer to the current head, and update head to this new node.
- **Pop:** Remove the node at the head, return its value, and update head to the next node. If the stack is empty, this results in an **underflow error**.
- **Top/Peek:** Return the value stored in the head node without removing it.
- **isEmpty:** Check whether head is `nullptr`.
- **Size:** Keep track of the number of nodes, or count dynamically by traversal.

Linked-List Stack: Complexity & Pros/Cons

- **Time Complexity:** `push`, `pop`, `top`, `isEmpty`, `size` are all $\Theta(1)$.
- **Space:** $\Theta(n)$ for nodes, plus an extra pointer field in each node.

Pros

- Stack can grow and shrink dynamically; no fixed capacity.
- No need for resizing as in arrays.

Cons

- Extra memory overhead for storing pointers.
- Poorer cache locality compared to arrays.
- Frequent dynamic allocations may introduce overhead or fragmentation.

Strategy: Valid Parentheses with a Stack

Goal: Decide if a string of brackets `()[]{}` is valid.

Idea

- Scan left to right; **push** every opening bracket onto the stack.
- On a closing bracket, **pop** and check that the popped opening matches the type.
- Any mismatch or attempting to pop from empty $\Rightarrow$ **invalid**.
- After processing all characters, the stack must be **empty** $\Rightarrow$ **valid**.

Edge Cases

- Extra closing bracket (underflow) $\Rightarrow$ invalid.
- Leftover openings (non-empty stack at end) $\Rightarrow$ invalid.

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read { $\rightarrow$ push {	void	$\langle (, \{ \rangle$

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read { $\rightarrow$ push {	void	$\langle (, \{ \rangle$
3	read [$\rightarrow$ push [	void	$\langle (, \{, [\rangle$

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read { $\rightarrow$ push {	void	$\langle (, \{ \rangle$
3	read [$\rightarrow$ push [	void	$\langle (, \{, [\rangle$
4	read] $\rightarrow$ pop	[	$\langle (, \{ \rangle$

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read { $\rightarrow$ push {	void	$\langle (, \{ \rangle$
3	read [$\rightarrow$ push [	void	$\langle (, \{, [\rangle$
4	read] $\rightarrow$ pop	[	$\langle (, \{ \rangle$
5	read } $\rightarrow$ pop	{	$\langle (\rangle$

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read { $\rightarrow$ push {	void	$\langle (, \{ \rangle$
3	read [$\rightarrow$ push [	void	$\langle (, \{, [\rangle$
4	read] $\rightarrow$ pop	[	$\langle (, \{ \rangle$
5	read } $\rightarrow$ pop	{	$\langle (\rangle$
6	read) $\rightarrow$ pop	(	$\langle \ \rangle$ (empty)

Example 1: Valid Parentheses

Input: `({[]})`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read { $\rightarrow$ push {	void	$\langle (, \{ \rangle$
3	read [$\rightarrow$ push [	void	$\langle (, \{, [\rangle$
4	read] $\rightarrow$ pop	[	$\langle (, \{ \rangle$
5	read } $\rightarrow$ pop	{	$\langle (\rangle$
6	read) $\rightarrow$ pop	(	$\langle \, \rangle$ (empty)
7	end of input & stack empty	**valid**	$\langle \, \rangle$

Example 2: Valid Parentheses

Input: `([)]`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$

Example 2: Valid Parentheses

Input: `([)]`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle (\rangle$
2	read [$\rightarrow$ push [	void	$\langle (, [\rangle$

Example 2: Valid Parentheses

Input: `([)]`

Step	Operation	Return	Updated Stack
1	read ($\rightarrow$ push (	void	$\langle ($
2	read [$\rightarrow$ push [	void	$\langle (, [$
3	read) $\rightarrow$ pop & compare	**mismatch**	$\langle (, [$ (unchanged)

Strategy: Infix → Postfix (Operator Stack)

Goal: Convert an infix expression to postfix using an **operator stack**.

Rules

- **Operands:** append directly to output.
- **Operators:** while the top of stack has *higher* precedence, or *equal and left-associative*, **pop** to output; then **push** the current operator.
- **Parentheses:** push `(`; on `)`, pop to output until `(` is removed.
- **End:** pop all remaining operators to output.

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	**Operation**	**Return**	**Op Stack**	**Postfix**
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	**Operation**	**Return**	**Op Stack**	**Postfix**
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $(\rightarrow$ push	void	$\langle *, (\rangle$	a

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	**Operation**	**Return**	**Op Stack**	**Postfix**
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $($ $\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	**Operation**	**Return**	**Op Stack**	**Postfix**
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $($ $\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	**Operation**	**Return**	**Op Stack**	**Postfix**
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $($ $\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab
6	read $c \rightarrow$ output	—	$\langle *, (, + \rangle$	abc

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $($ $\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab
6	read $c \rightarrow$ output	—	$\langle *, (, + \rangle$	abc
7	read $)$ $\rightarrow$ pop until $($	$+$	$\langle * \rangle$	$abc+$

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\,\rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read ($\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab
6	read $c \rightarrow$ output	—	$\langle *, (, + \rangle$	abc
7	read) $\rightarrow$ pop until (	$+$	$\langle * \rangle$	$abc+$
8	read / $\rightarrow$ pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$abc + *$

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read ($\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab
6	read $c \rightarrow$ output	—	$\langle *, (, + \rangle$	abc
7	read) $\rightarrow$ pop until (	$+$	$\langle * \rangle$	$abc+$
8	read / $\rightarrow$ pop higher/equal $(*)$	$*$	$\langle\ \rangle$	$abc + *$
9	then push /	void	$\langle / \rangle$	$abc + *$

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\,\rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $($ $\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab
6	read $c \rightarrow$ output	—	$\langle *, (, + \rangle$	abc
7	read $)$ $\rightarrow$ pop until $($	$+$	$\langle * \rangle$	$abc+$
8	read $/ \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$abc + *$
9	then push $/$	void	$\langle / \rangle$	$abc + *$
10	read $d \rightarrow$ output	—	$\langle / \rangle$	$abc + *d$

Example 1: Infix $\rightarrow$ Postfix

Infix: $a * (b + c)/d$ Target Postfix: $abc + *d/$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\,\rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $($ $\rightarrow$ push	void	$\langle *, (\rangle$	a
4	read $b \rightarrow$ output	—	$\langle *, (\rangle$	ab
5	read $+ \rightarrow$ push	void	$\langle *, (, + \rangle$	ab
6	read $c \rightarrow$ output	—	$\langle *, (, + \rangle$	abc
7	read $)$ $\rightarrow$ pop until $($	$+$	$\langle * \rangle$	$abc+$
8	read $/ \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$abc + *$
9	then push $/$	void	$\langle / \rangle$	$abc + *$
10	read $d \rightarrow$ output	—	$\langle / \rangle$	$abc + *d$
11	end $\rightarrow$ pop remaining	$/$	$\langle\,\rangle$	$abc + *d/$

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\,\rangle$	a

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\,\rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab
4	read $- \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$ab*$

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab
4	read $- \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\ \rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab
4	read $- \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\ \rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read $($ $\rightarrow$ push	void	$\langle -, (\rangle$	$ab*$

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\,\rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab
4	read $- \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read $($ $\rightarrow$ push	void	$\langle -, (\rangle$	$ab*$
7	read $c \rightarrow$ output	—	$\langle -, (\rangle$	$ab * c$

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab
4	read $- \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\ \rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read $($ $\rightarrow$ push	void	$\langle -, (\rangle$	$ab*$
7	read $c \rightarrow$ output	—	$\langle -, (\rangle$	$ab * c$
8	read $- \rightarrow$ push	void	$\langle -, (, - \rangle$	$ab * c$

Example 2: Infix $\rightarrow$ Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read $a \rightarrow$ output	—	$\langle\ \rangle$	a
2	read $* \rightarrow$ push	void	$\langle * \rangle$	a
3	read $b \rightarrow$ output	—	$\langle * \rangle$	ab
4	read $- \rightarrow$ pop higher/equal $(*)$	$*$	$\langle\ \rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read $($ $\rightarrow$ push	void	$\langle -, (\rangle$	$ab*$
7	read $c \rightarrow$ output	—	$\langle -, (\rangle$	$ab * c$
8	read $- \rightarrow$ push	void	$\langle -, (, - \rangle$	$ab * c$
9	read $d \rightarrow$ output	—	$\langle -, (, - \rangle$	$ab * cd$

Example 2: Infix → Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read a → output	—	$\langle\,\rangle$	a
2	read $*$ → push	void	$\langle * \rangle$	a
3	read b → output	—	$\langle * \rangle$	ab
4	read $-$ → pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read (→ push	void	$\langle -, (\rangle$	$ab*$
7	read c → output	—	$\langle -, (\rangle$	$ab * c$
8	read $-$ → push	void	$\langle -, (, - \rangle$	$ab * c$
9	read d → output	—	$\langle -, (, - \rangle$	$ab * cd$
10	read) → pop until (	$-$	$\langle - \rangle$	$ab * cd-$

Example 2: Infix → Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read a → output	—	$\langle\,\rangle$	a
2	read $*$ → push	void	$\langle * \rangle$	a
3	read b → output	—	$\langle * \rangle$	ab
4	read $-$ → pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read (→ push	void	$\langle -, (\rangle$	$ab*$
7	read c → output	—	$\langle -, (\rangle$	$ab * c$
8	read $-$ → push	void	$\langle -, (, - \rangle$	$ab * c$
9	read d → output	—	$\langle -, (, - \rangle$	$ab * cd$
10	read) → pop until (	$-$	$\langle - \rangle$	$ab * cd-$
11	read ^ → push	void	$\langle -, \hat{} \rangle$	$ab * cd-$

Example 2: Infix → Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{} -$

Step	Operation	Return	Op Stack	Postfix
1	read a → output	—	$\langle\,\rangle$	a
2	read $*$ → push	void	$\langle * \rangle$	a
3	read b → output	—	$\langle * \rangle$	ab
4	read $-$ → pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read $($ → push	void	$\langle -, (\rangle$	$ab*$
7	read c → output	—	$\langle -, (\rangle$	$ab * c$
8	read $-$ → push	void	$\langle -, (, - \rangle$	$ab * c$
9	read d → output	—	$\langle -, (, - \rangle$	$ab * cd$
10	read $)$ → pop until $($	$-$	$\langle - \rangle$	$ab * cd-$
11	read $\hat{}$ → push	void	$\langle -, \hat{} \rangle$	$ab * cd-$
12	read e → output	—	$\langle -, \hat{} \rangle$	$ab * cd - e$

Example 2: Infix → Postfix

Infix: $(a * b) - (c - d)^e$ Target Postfix: $ab * cd - e\hat{}-$

Step	Operation	Return	Op Stack	Postfix
1	read a → output	—	$\langle\,\rangle$	a
2	read $*$ → push	void	$\langle * \rangle$	a
3	read b → output	—	$\langle * \rangle$	ab
4	read $-$ → pop higher/equal $(*)$	$*$	$\langle\,\rangle$	$ab*$
5	then push $-$	void	$\langle - \rangle$	$ab*$
6	read $($ → push	void	$\langle -, (\rangle$	$ab*$
7	read c → output	—	$\langle -, (\rangle$	$ab * c$
8	read $-$ → push	void	$\langle -, (, - \rangle$	$ab * c$
9	read d → output	—	$\langle -, (, - \rangle$	$ab * cd$
10	read $)$ → pop until $($	$-$	$\langle - \rangle$	$ab * cd-$
11	read $\hat{}$ → push	void	$\langle -, \hat{} \rangle$	$ab * cd-$
12	read e → output	—	$\langle -, \hat{} \rangle$	$ab * cd - e$
13	end → pop remaining	$\hat{}$, $-$	$\langle\,\rangle$	$ab * cd - e\hat{}-$